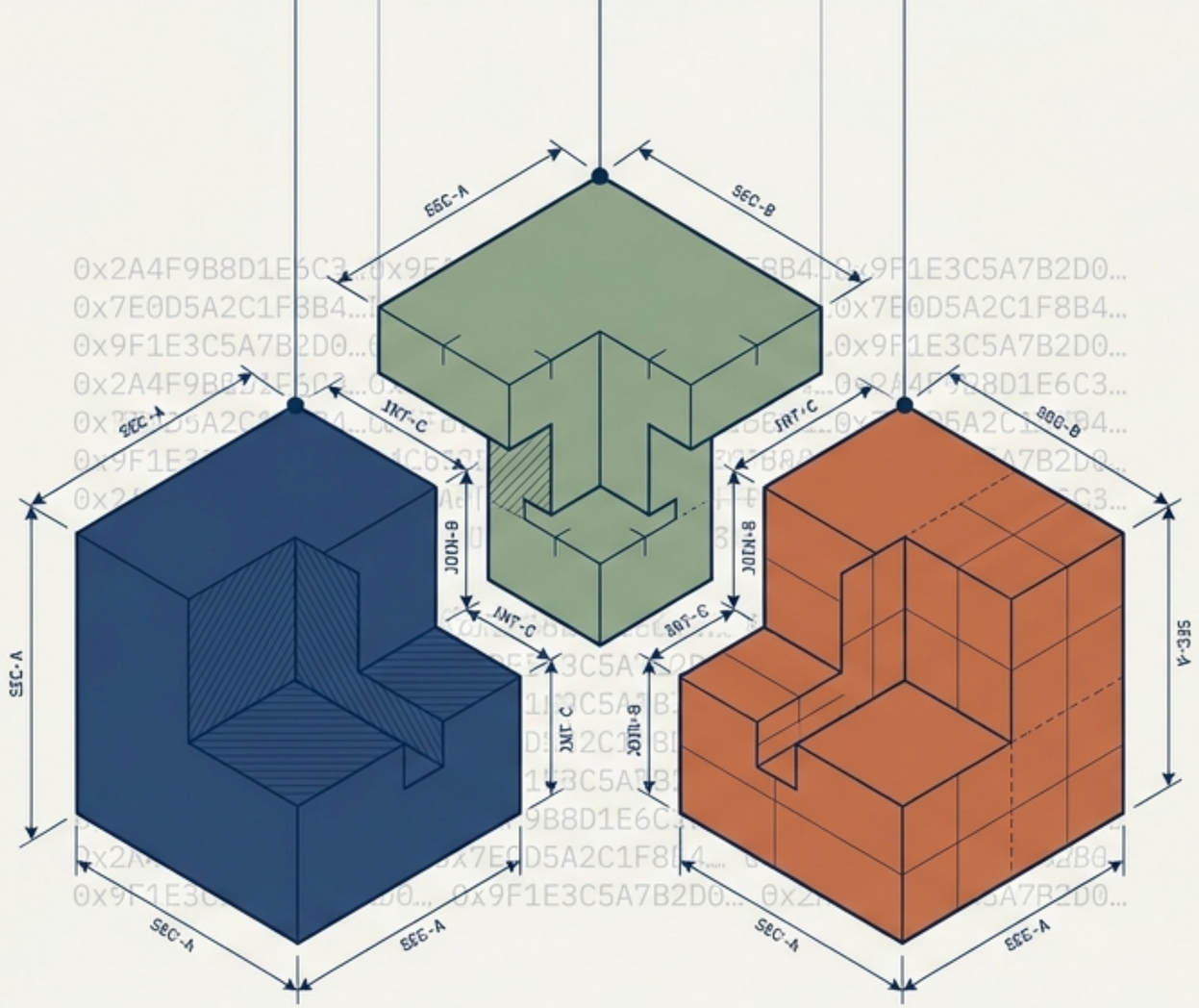


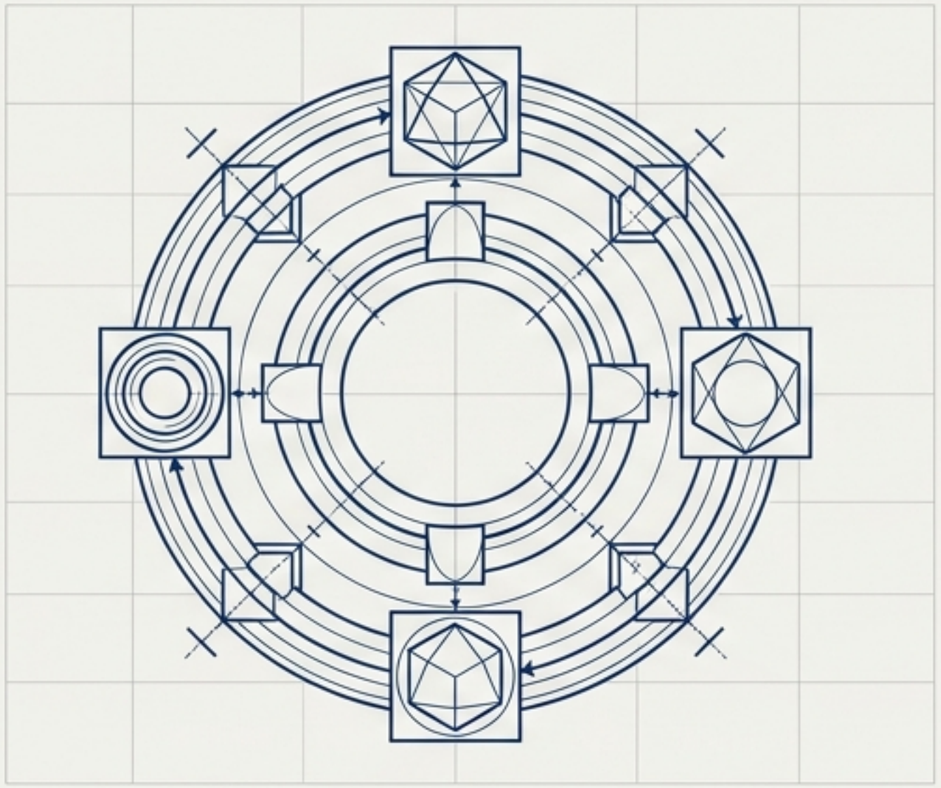
Proof-Carrying Autonomous Systems

Integrating Theorem, selfdriven, and VCGs for verifiable AI governance.

Architecture and Research Assessment

Gemini Notebook

Correctness is not the same as legitimacy.



Correctness: Does the implementation satisfy its formal specification?



Legitimacy: Is the context evidenced, authorized, and appropriate for human agency?

Formal verification proves a controller does what it is designed to do. Contextual governance determines whether that design is justified. Both are required for autonomous systems.

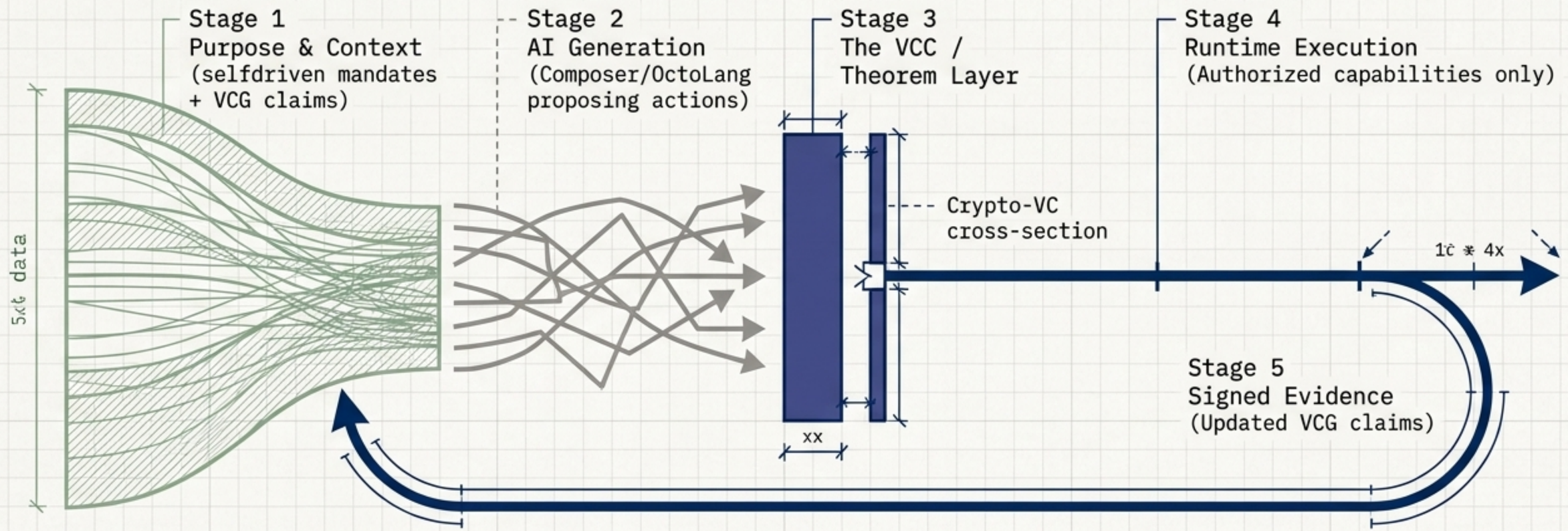
Gemini Notebook

The Autonomous Trust Stack

Component	Primary Question	Core Metric	Verification Target
Theorem	Can we mathematically prove software behaviors?	**Correctness**	Controller specification & implementation.
VCC (Verified Code Controller)	Does the control mechanism enforce authorized boundaries?	**Determinism**	Policy limits, fail-closed logic.
VCG (Verified Context Graph)	What is known and where did the evidence come from?	**Provenance**	Signature validation, graph consistency.
Octonomous	Can AI operate persistently within governance rules?	**Authority**	Delegation rules, state transitions.
selfdriven	Does this support community self-actuation?	**Legitimacy**	Protocol invariants, trusted interoperability.

Gemini Notebook

The Executional Funnel: From Context to Proven Action



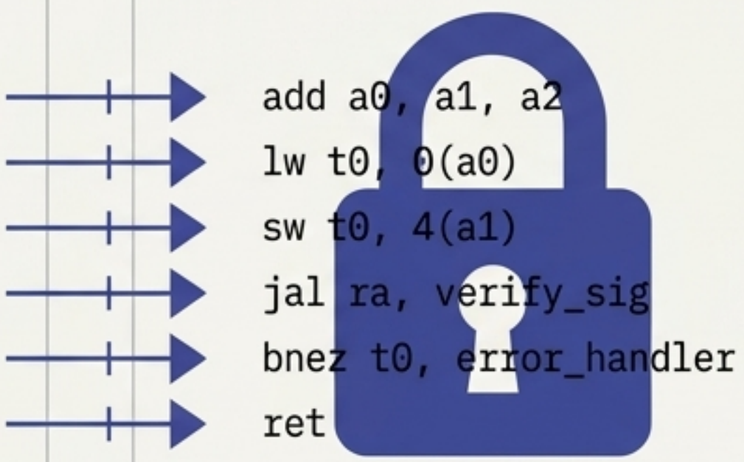
Note: The critical separation is between proving an implementation correct (Theorem) and proving an action is appropriate in context (VCGs).

Gemini Notebook

Theorem: Independent Provability at the Instruction Level

Code Compilation

```
function analyze_data(input_stream):
    verify_signature(input_stream)
    parse_header(input_stream)
    process_payload(input_stream)
    return certified_result
end function
```



```
add a0, a1, a2
lw t0, 0(a0)
sw t0, 4(a1)
jal ra, verify_sig
bnez t0, error_handler
ret
```

MACHINE-CHECKABLE MATHEMATICAL PROOF

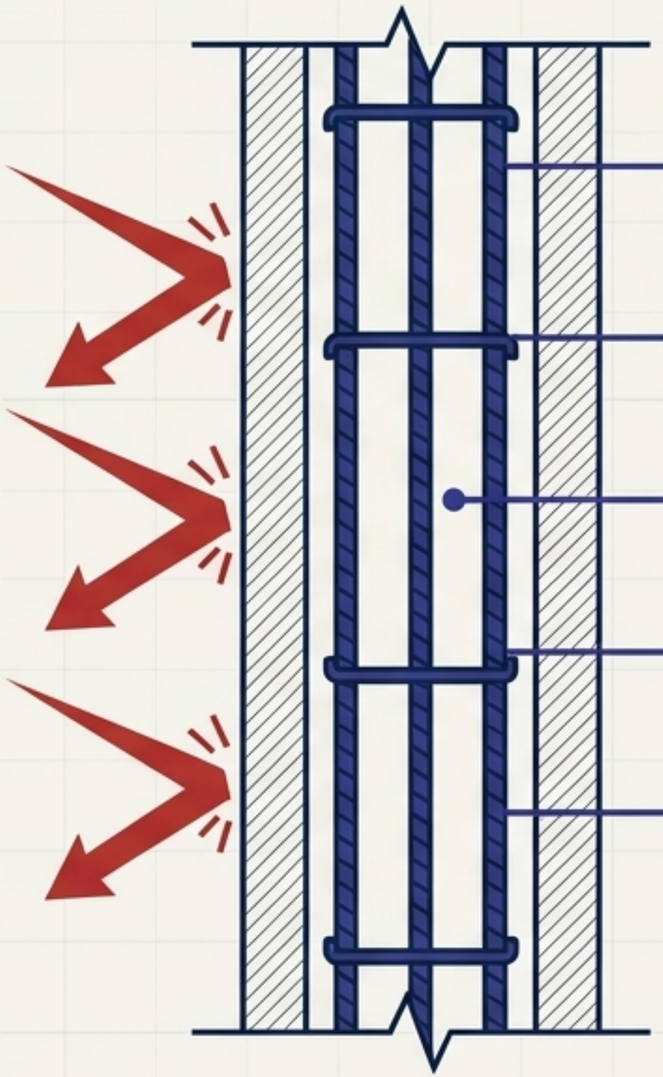
October 2026: Anthropic collaboration demonstrated verification of 239 entry points across 123 open-source libraries.

Proof Standard: Rocq proof checker targeting static RISC-V binaries (no-crash guarantees for inputs up to 64 MiB).

Scale: Progressively verifying dependency dependency ecosystems to allow proof reuse across components.

Gemini Notebook

The Verified Code Controller (VCC) as Security Membrane



Unverified Actions
Invalid inputs, expired VCGs, unauthorized attempts

Theorem Formal Proofs

1. Action cannot exceed delegated authority.
2. Expired VCG snapshots immediately halt execution.
3. Missing evidence strictly results in DENY or ESCALATE.
4. Consumed authorizations cannot be replayed.
5. Controller cannot broaden its own permissions.

Gemini Notebook

The Artifact: Proof-Carrying VCCs

Digital Dossier


```
{
  "type": "selfdriven.vcc.proof",
  "controller": "vcc.authority-gate",
  "version": "0.1.0",
  "artifactHash": "sha256:...",
  "specificationHash": "sha256:...",
  "proofHash": "sha256:...",
  "proofSystem": "rocq",
  "properties": [
    "authority-confinement",
    "fail-closed",
    "replay-protection",
    "context-validity"
  ],
  "verificationStatus": "VERIFIED"
}
```

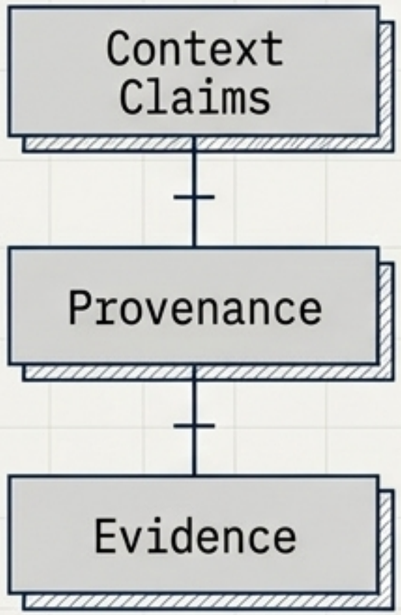

Cryptographically binds the exact build and dependencies to the verification status.

Formal specifications the controller has been mathematically proven to uphold.

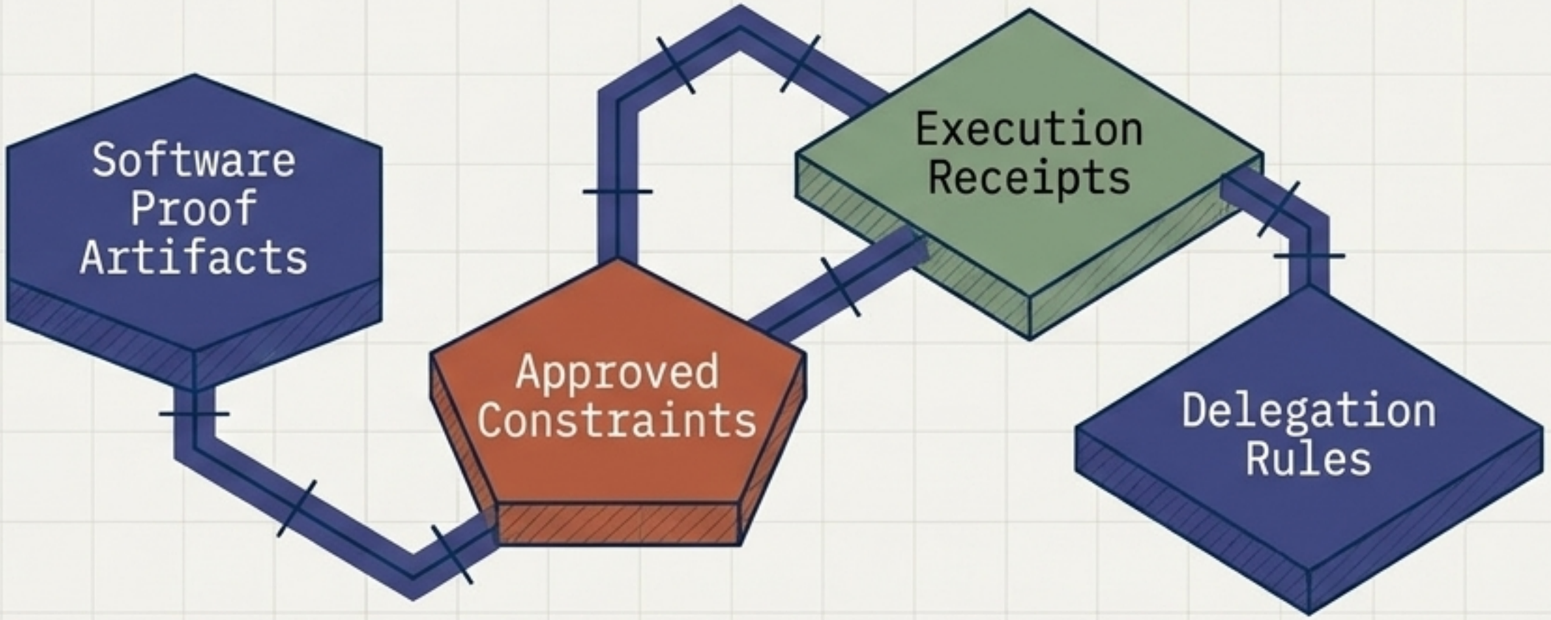
Gemini Notebook

Expanding VCGs: From Context Chains to Proof Graphs

Today



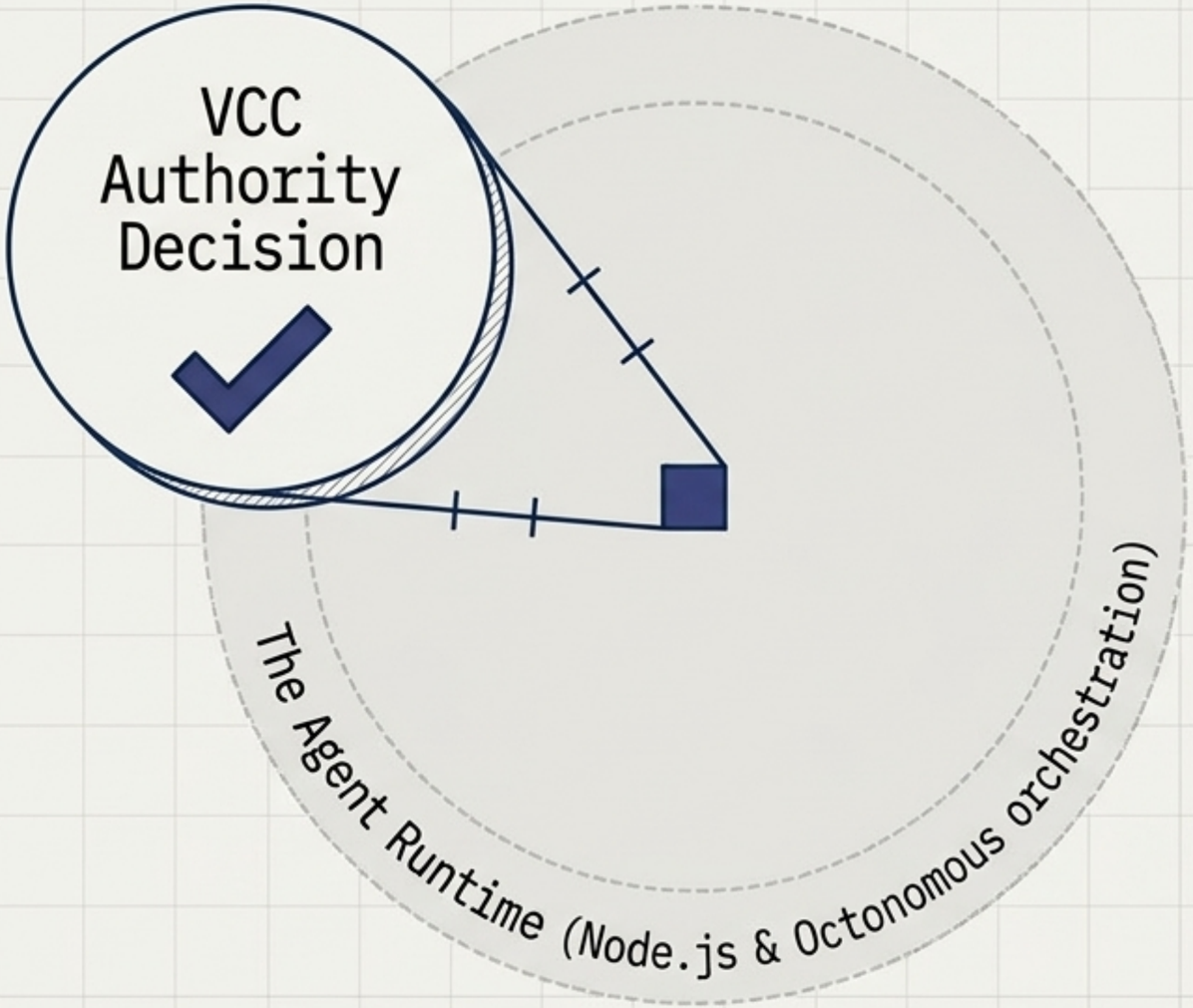
The Future - The Proof Graph



A community can independently check another community's controller proofs via the graph, without needing to trust the intelligence that generated the controller.

Gemini Notebook

Pragmatic Integration: Verify the Bottleneck, Not the Ocean



1. Keep broad orchestration in existing Node.js/Octonomous runtimes.
2. Isolate security-critical authority decisions into a small, precise module.
3. Implement this bottleneck in a verification-friendly language.
4. Formally prove authority & fail-closed invariants via Theorem.
5. Bind proofs to release artifacts within the VCG.

Gemini Notebook



Proof-Carrying Autonomous Systems

Intelligence can generate and evolve behavior, but an independently verifiable trust architecture must control what may actually happen.

1

Make formal verification a first-class property of controllers.

2

Use VCGs as the shared evidence structure to connect proofs to real-world outcomes.

3

Result: Autonomy anchored in both mathematical correctness and contextual legitimacy.

Gemini Notebook